

FoundIt — Live Demo Script

Length: about 10 minutes, plus questions. **URL:** <https://found-it-ashy-sigma.vercel.app> **Needs:** nothing beyond the free Spark plan. No Cloud Functions, no Google popups: every step uses the three demo accounts and the seeded data.

This script is rehearsed automatically. `web/test/demo-script.mjs` performs every click and types every value below, in this order, on three screens. Step numbers match. If you change a step here, change it there too.

Screens and roles

Screen	Signed in as	Driven by	Shown to the room
A	Riya Singh (student)	Nidhi, who narrates	Projector
B	Arjun Nair (student)	Shenza	Second display, or turned to the room
C	Meera Das (moderator)	Shanid	When it's his turn

Hadi narrates the marketplace (section 6) while Nidhi and Shenza click.

Use three **separate** browsers or Chrome profiles. Two incognito windows share one sign-in, so they won't work as two screens.

0. Pre-flight (30 minutes before, not in the room)

First time only: follow **docs/DEMO-SETUP.md** (deploy the rules, create the demo accounts, turn on the demo sign-in buttons).

Then before every run:

```
export GOOGLE_APPLICATION_CREDENTIALS=~/.foundit-key.json
node scripts/seed-demo-data.mjs --project foundit-fcfcc --wipe-all
```

This resets the database to exactly the state the script expects. `--wipe-all` also deletes any real posts; leave it off if you want to keep them, but then the counts in step 2 will differ.

- ☐ Screen **A**: site open, **signed out**, hard-reloaded
- ☐ Screen **B**: signed in as **Arjun Nair** (Sign in → *Arjun Nair*)
- ☐ Screen **C**: signed in as **Meera Das**
- ☐ Screen A mirrored to the projector
- ☐ One full rehearsal on the room's wifi, then reseed
- ☐ Phone hotspot ready in case the campus wifi misbehaves

1. The problem (Nidhi, 45 s, no clicking)

"Every semester our campus loses hundreds of things: ID cards, calculators, water bottles. Getting them back means a 300-person WhatsApp group with no memory. Seniors leaving campus sell cycles and books through the same chaos. FoundIt puts both in one place that only verified students can use."

"The interesting part isn't the UI. Lost-and-found is event-driven, and trust has to be enforced by the server, not the client. That's what we'll show."

2. The feed (Nidhi on A, 90 s)

1. **Signed out**, point at the feed: **23 posts** of three kinds (lost, found, for sale). Anyone can browse; only verified students can act.
2. Click the **Lost & Found** tab (13 posts), then **Marketplace** (10), then back to **All**.
3. Type **casio** in the search box. Three cards are left: a lost calculator, a found one and one for sale. Clear the search.
4. On **Casio Scientific Calculator in LH-204**, click **Claim this**. You are signed out, so the sign-in dialog opens.
5. Click **Riya Singh**. You're signed in.

"Demo accounts are real Firebase users with verified emails. The security rules treat them exactly like students; nothing is bypassed. In production, sign-in is Google with the campus domain."

3. Real-time sync (Nidhi on A, Shenza on B, 90 s)

The strongest moment. Keep screen B visible and **don't reload it**.

1. On **A**, click **Post**. It opens on **Lost item**. Fill in:
 - Title: **Lost blue spiral lab record notebook**
 - Category: **Books & Notes**
 - Location: **Lecture Halls (LH)**
 - Description: **Blue spiral-bound lab record, name on the inside cover, last used in the LH-101 electronics lab.**

Click **Post lost report**.

2. The card appears on **screen B** by itself.

"No refresh. Every open client listens to the database, and Firestore pushes each change to them. It's the Observer pattern, and it's why we picked a realtime document store over a REST API."

4. Smart matching (Shenza on B, 60 s)

1. On **B**, click **Post**, choose **Found item**, and fill in:

- Title: Found blue spiral lab record book in LH-101
- Category: **Books & Notes**
- Location: **Lecture Halls (LH)**
- Description: Picked up a blue spiral lab record after the electronics lab in LH-101.

Click **Post found report**.

2. Both notebook cards get a **% match** badge, on both screens.
3. On **A**, click the badge on Riya's lost notebook. The breakdown names Arjun's found post. Close it.

"The score weighs five factors: keyword overlap (Jaccard), category, campus zone with adjacency, time, and the poster's trust. The Cloud Function version computes it on the server when a post is written, and the client computes the same formula when the function isn't deployed."

5. Claim, verify, return (Nidhi on A, Shenza on B, 2 min)

Riya lost the notebook, so Riya claims it. Arjun found it, so Arjun approves.

1. On **A**, on Arjun's **Found blue spiral lab record book in LH-101**, click **Claim this**. Point out the first field: it already offers Riya's own **lost** notebook with its match score, because she posted one. Leave it selected. In the proof box type: Name written inside the front cover: Riya Singh, ECE lab batch 4 Click **Submit Claim Request**. The confirmation points her to **My claims**. Close it. Her card now reads **Claim sent · waiting**, so she can't claim twice.
2. On **B**, without reloading, Arjun's card now reads **Review claims (1)**. Click it. Arjun sees his own description at the top — *private to you* — then Riya's proof, then **Linked to their own report: ... % match**. Read the proof aloud and compare.

"No notification needed: the count is on the finder's card as soon as the claim lands. Two things make the proof mean something. It's private, so only the finder, the claimant and moderators can read it. And the item's own description is private too — that's the part people miss. If the card showed 'name written inside the front cover', a scammer would just read it and type it back. So the card shows the title and coarse tags; the identifying detail lives in a subdocument only the poster can read."

"The link under the proof is checked by the rules, not the UI: the report she cited has to be one she actually posted, of the opposite type. Nobody can point at a stranger's matching post to look credible."

3. Still on **B**, click **Message** under Riya's claim and send Found it! LH-101 at 4 pm?

4. On **A**, click the **Message** icon on the same card. Arjun's message is there. Reply **Perfect, see you there.** It appears on B at once. Close both chats.
5. On **B**, click **Review claims (1) → Approve & mark returned**. The dialog reads *Returned. This report is closed.* Close it. The card reads **Returned** on every screen.
6. On **A**, open the account menu → **My claims: Approved and marked returned**. Close it.

"Marking it returned is one atomic write. The claim's approval and the item's status land together or not at all, and the rules refuse to close an item unless the claim it names really is approved. If other people had claimed it, the same write declines them."

6. Marketplace deal (Hadi narrates; Nidhi on A, Shenza on B, 90 s)

The seed has Riya already offering ₹3000 for Arjun's cycle.

1. On **A**, open the **Marketplace** tab. On **Hero Sprint Cycle**, click **Make a deal**. It shows *Offer sent to Arjun Nair*. Click **Confirm deal: Waiting for Arjun Nair to confirm**. Close it.
2. On **B**, open **Marketplace**. The cycle card now reads **Confirm sale**. Click it (it shows *Riya Singh offered ₹3000*), then **Confirm sale: Transaction complete**. Close it.
3. On **A**, the cycle card now reads **Sold · Rate seller**. Click it, leave the five stars, type **Smooth handover, cycle exactly as described.** and click **Submit review**. Close it.

"Neither side can mark it sold alone. Each party writes only their own confirmation, and the rules allow the flip to sold only once both are stored. It's a two-party handshake (a small saga), with the security rules as the coordinator."

7. AI search (Shanid on C, 45 s)

1. On **C**, click **AI Search**, type **I lost my ID card**, and send it.
2. It returns **Campus ID Card — Ananya R.**, not a random item. Close it.

"With a Gemini key the query goes to the model along with the candidate posts. Without one, or if the API fails, it falls back to a local ranked keyword search. That's the Strategy pattern: the feature degrades instead of disappearing."

8. Flag and moderate (Shenza on B, Shanid on C, 60 s)

1. On **B**, on **iPhone 15 Pro — ₹5000 urgent**, click the **flag** icon. Choose **Suspected scam or fake report** and submit.
2. On **C**, open the account menu → **Moderation queue**. The iPhone listing is there, reported twice (the seed has one earlier report).
3. Click **Remove post**. Both reports leave the queue, and the listing disappears from every screen.

"Flags are readable only by moderators. That's enforced by the rules, not by hiding a button. With Cloud Functions deployed, removing a post also gives its owner a strike and lowers their trust score."

9. Trust (Nidhi on A, 30 s)

Open the account menu → **My Profile**. Point at Riya's **Trust Score** and **Avg Rating**.

"Trust is Bayesian with damping: $(\text{avg} - 3) \times \text{count} / (\text{count} + 5) \times 40$. One five-star review from a friend barely moves it; many good exchanges do. No client can write its own trust. The rules forbid it, and only the server computes it."

10. Architecture close (Nidhi, 60 s)

Put the architecture slide up.

"An event-driven core. A denormalised read model so the feed needs no joins. Security rules as the gateway for every write. And graceful degradation at every external dependency: AI, matching and deals all keep working without their cloud functions."

Stop talking and take questions.

If something breaks

Symptom	Do this
No demo sign-in buttons	<code>VITE_DEMO_AUTH</code> / <code>VITE_DEMO_PASSWORD</code> are not set in Vercel, or it wasn't redeployed. Sign in with Google instead.
Counts in step 2 are off	The seed wasn't re-run, or was run without <code>--wipe-all</code> . Say "about two dozen posts" and carry on.
"Missing or insufficient permissions"	The rules aren't deployed. Run <code>firebase deploy --only firestore:rules,firestore:indexes</code> .
A screen didn't update	Reload that screen once. Never debug live.
Anything else	Say what should happen, move on, and offer to show it afterwards.

Likely questions

- *Why Firestore over SQL?* Realtime subscriptions, an offline cache, and rules as a declarative authorization layer. The costs are denormalisation and no joins.
- *How do you stop someone inflating their own trust?* Clients can't write it. The rules guard the field with `keeps()`, and only server code (the Admin SDK) bypasses the rules.

- *Can someone claim an item twice?* No. A claim is stored under the claimant's own ID, and the rules refuse a second one.
- *Why aren't the functions deployed?* Cloud Functions v2 needs the Blaze plan. They're tested against the emulator (28 integration tests), and every user-facing flow has a fallback that works without them.
- *How is this tested?* Seven suites, over 200 checks, including this exact script run as a test (docs/TESTING.md).